

- **Music Streaming Database System**

- Ivana Aguilar (017788628)
- Hodo Abdulkarim (018106751)
- Atay Moya (018051930)

- **Introduction:**

- Brief Project Overview: We aim to create a music streaming database that allows for efficient storage and retrieval of users, artists, albums, songs, playlists, playlist songs, likes, and listening history. The system will manage user accounts, track listening history, and support recommendations based on likes and listening history.
- Purpose: This system will simplify music data management and enhance the user experience through efficient search and playback tracking. It will help users easily access, organize, and stream music with accurate records.

- **Objectives:**

- Primary Goals: Build a database to store information for users to easily search, stream, and manage their music. It will record user preferences and listening history to optimize query performance.
- Functionality Overview: Allow users to search for songs, artists, and albums, create and manage playlists, and stream music. The system will also provide functions to add, update, and delete music records.

- **Proposed Database Schema:**

- Entities:
 - Users: All the users who use the platform
 - Artists: All the artists who post music on the platform
 - Albums: All the albums that artists on the platform have released
 - Songs: Every song that exists on the platform
 - Playlists: Every playlist that exists on the platform
 - PlaylistSongs: The songs on a given playlist
- Attributes:
 - Users: username, user_id, email
 - Artists: artistname, artist_id
 - Albums: album_id, title, release_year, artist_id
 - Songs: song_id, title, duration, album_id, genre
 - Playlists: playlist_id, user_id, playlist_name, created_date
 - Playlist songs: playlist_id, song_id, added_date
- Relationships:
 - Many to Many (M:N)
 - Playlists ↔ Songs (a song can be on multiple playlists, a playlist can have multiple songs)

- Artist ↔ Album (An artist can have many albums, and an album can have multiple artists)
 - One to Many (1:N)
 - User → Queue
 - Song → Queue
 - User → Playlist (One user can have multiple playlists but a playlist can only belong to one user)
 - Song → Queue (A song can be in multiple user's queues)
 - Album → Song (An album can have multiple songs but a song can only be on one album)
- **Functional Requirements:**
 - User Description: Users will be people of all age groups and demographics who wish to use a system that allows them to listen to music, and they will interact with the system to browse songs, create playlists, and like songs.
 - Key Functionalities: Create playlists, Share Playlists, Add new users, Add new songs/playlists, Display Songs/Playlists
- **Non-Functional Requirements:**
 - Accessibility and Security:
 - Middleware/Rate Limiting
- **Tools and Technologies:**
 - Database and Technology Stack:
 - Python
 - MySQL (<https://dev.mysql.com/downloads/connector/j/>)
 - Ruff Linter
 - Development Environment:
 - VSCode
- **Conclusion:**
 - Expected Outcomes: A fully functional music streaming database system that meets all specified requirements by efficiently storing and retrieving music data.
 - Next Steps: Create a timetable showing the project tasks, deliverables, ownerships, proposed due dates, and check marks for items completed. Define the database schema in more detail and start developing the application.